

Python Programming

Day 3: 자료구조

리스트, 튜플, 딕셔너리, 집합, 컴프리헨션

Contents

1	학습 목표	2
2	리스트	2
2.1	인덱싱	2
2.2	슬라이싱	2
2.3	원소 수정	3
3	리스트 메서드	3
3.1	원소 추가: <code>append()</code> 와 <code>insert()</code>	3
3.2	원소 삭제: <code>remove()</code> 와 <code>pop()</code>	3
3.3	정렬과 순서 뒤집기	4
3.4	개수와 위치 찾기	4
4	튜플	5
4.1	인덱싱과 슬라이싱	5
4.2	튜플 언패킹	5
4.3	원소가 하나인 튜플	5
5	딕셔너리	6
5.1	값에 접근하기	6
5.2	추가와 수정	6
5.3	삭제	6
5.4	안전한 조회: <code>get()</code>	6
5.5	딕셔너리 순회	7
6	집합	7
6.1	리스트의 중복 제거	7
6.2	원소 추가와 삭제	7
6.3	집합 연산	8
6.4	소속 검사	8
7	컴프리헨션	8
7.1	리스트 컴프리헨션	8
7.2	조건을 포함한 리스트 컴프리헨션	9
7.3	조건 표현식 사용	9
7.4	딕셔너리 컴프리헨션	9
7.5	집합 컴프리헨션	9
8	자료구조 비교	10
9	종합 실습: <code>practice_day03.py</code>	10
9.1	학생 목록과 중복 제거	10
9.2	점수 딕셔너리	10
9.3	보너스 점수 계산	10
9.4	합격자 선별	10
9.5	최고 점수와 최고 득점자	10
9.6	튜플로 결과 묶기	11
9.7	전체 코드	11
10	실습 파일 목록	12
11	핵심 요약	12

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- 리스트를 생성하고 인덱싱과 슬라이싱으로 원소에 접근하는 방법
- 리스트의 주요 메서드를 이용해 데이터를 추가, 수정, 삭제하는 방법
- 튜플의 불변성과 언패킹의 의미
- 딕셔너리에서 키와 값을 저장하고 순회하는 방법
- 집합으로 중복을 제거하고 집합 연산을 수행하는 방법
- 리스트, 딕셔너리, 집합 컴프리헨션을 작성하는 방법
- 여러 자료구조를 결합하여 학생 성적 데이터를 처리하는 방법

2. 리스트

리스트는 여러 값을 하나의 변수에 순서대로 저장하는 자료형이다. 대괄호를 사용하며, 서로 다른 자료형도 함께 저장할 수 있다.

```
1 numbers = [10, 20, 30, 40]
2 names = ["Kim", "Lee", "Park"]
3 mixed = [1, "Python", 3.14]
```

리스트의 각 원소에는 0부터 시작하는 인덱스가 붙는다.

2.1. 인덱싱

```
1 numbers = [10, 20, 30, 40]
2
3 print(numbers[0])
4 print(numbers[2])
5 print(numbers[-1])
```

출력:

```
10
30
40
```

음수 인덱스는 리스트의 뒤에서부터 원소를 센다. 따라서 -1은 마지막 원소를 의미한다.

2.2. 슬라이싱

슬라이싱은 리스트의 일부 구간을 새로운 리스트로 가져오는 방법이다.

```
1 numbers = [10, 20, 30, 40, 50]
2
3 print(numbers[1:4])
4 print(numbers[:3])
5 print(numbers[2:])
```

출력:

```
[20, 30, 40]
[10, 20, 30]
[30, 40, 50]
```

start:stop에서 시작 인덱스는 포함되고 끝 인덱스는 포함되지 않는다.

2.3. 원소 수정

리스트는 생성한 뒤에도 원소를 바꿀 수 있는 가변 객체이다.

```
1 numbers = [10, 20, 30]
2
3 numbers[1] = 100
4
5 print(numbers)
```

출력:

```
[10, 100, 30]
```

핵심 포인트

리스트는 순서를 가지며, 중복을 허용하고, 생성 이후에도 원소를 수정할 수 있다.

실습

01_list.py에서 다음 리스트를 만들고 첫 번째 원소, 마지막 원소, 두 번째부터 네 번째 원소까지의 슬라이스를 출력한다.

```
1 fruits = ["Apple", "Banana", "Orange", "Grape"]
```

3. 리스트 메서드

리스트는 데이터를 추가하거나 삭제하고 순서를 바꾸기 위한 여러 메서드를 제공한다.

3.1. 원소 추가: append()와 insert()

append()는 리스트 끝에 원소를 추가한다.

```
1 numbers = [1, 2, 3]
2 numbers.append(4)
3
4 print(numbers)
```

insert(index, value)는 지정한 위치에 원소를 삽입한다.

```
1 numbers = [1, 2, 3]
2 numbers.insert(1, 100)
3
4 print(numbers)
```

3.2. 원소 삭제: remove()와 pop()

remove(value)는 값과 일치하는 첫 번째 원소를 삭제한다.

```

1 numbers = [10, 20, 30, 20]
2 numbers.remove(20)
3
4 print(numbers)

```

pop()은 원소를 삭제하면서 그 값을 반환한다. 인덱스를 생략하면 마지막 원소를 삭제한다.

```

1 numbers = [10, 20, 30]
2 removed = numbers.pop()
3
4 print(removed)
5 print(numbers)

```

핵심 포인트

remove()는 값을 기준으로 삭제하고, pop()은 인덱스를 기준으로 삭제하면서 삭제한 값을 반환한다.

3.3. 정렬과 순서 뒤집기

```

1 numbers = [5, 2, 8, 1]
2
3 numbers.sort()
4 print(numbers)
5
6 numbers.reverse()
7 print(numbers)

```

sort()는 리스트 자체를 정렬하고, reverse()는 현재 순서를 뒤집는다.

3.4. 개수와 위치 찾기

```

1 numbers = [1, 2, 2, 2, 3]
2
3 print(numbers.count(2))
4 print(numbers.index(3))

```

메서드	기능
append(value)	마지막에 원소 추가
insert(index, value)	지정한 위치에 원소 삽입
remove(value)	일치하는 첫 번째 값 삭제
pop(index)	원소를 삭제하고 반환
sort()	오름차순 정렬
reverse()	현재 순서 뒤집기
count(value)	값의 개수 반환
index(value)	첫 번째 위치 반환

실습

02_list_methods.py에서 빈 리스트를 만든 뒤 10, 20, 30을 차례로 추가하고, 20을 삭제한 후 최종 리스트를 출력한다.

4. 튜플

튜플은 리스트와 마찬가지로 여러 값을 순서대로 저장하지만, 생성 이후 원소를 수정할 수 없는 불변 객체이다.

```
1 numbers = (10, 20, 30)
2 student = ("Kim", 95)
```

4.1. 인덱싱과 슬라이싱

튜플도 리스트와 같은 방식으로 인덱싱과 슬라이싱을 지원한다.

```
1 numbers = (10, 20, 30, 40)
2
3 print(numbers[0])
4 print(numbers[-1])
5 print(numbers[1:3])
```

그러나 다음처럼 원소를 수정하려 하면 오류가 발생한다.

```
1 numbers = (10, 20, 30)
2 numbers[0] = 100
```

4.2. 튜플 언패킹

튜플의 각 값을 여러 변수에 한 번에 대입할 수 있다.

```
1 student = ("Kim", 95)
2 name, score = student
3
4 print(name)
5 print(score)
```

두 변수의 값을 교환할 때도 언패킹을 활용할 수 있다.

```
1 a = 10
2 b = 20
3
4 a, b = b, a
5
6 print(a)
7 print(b)
```

4.3. 원소가 하나인 튜플

원소가 하나인 튜플은 값 뒤에 쉼표를 붙여야 한다.

```
1 value = (10,)
2 print(type(value))
```

핵심 포인트

튜플을 구분하는 핵심은 괄호 자체가 아니라 쉼표이다. (10)은 정수지만 (10,)은 튜플이다.

실습

03_tuple.py에서 ("Alice", 100)을 튜플로 저장하고 언패킹하여 이름과 점수를 각각 출력한다.

5. 딕셔너리

딕셔너리는 데이터를 키와 값의 쌍으로 저장한다. 리스트가 위치를 이용해 원소에 접근한다면, 딕셔너리는 의미 있는 키를 이용한다.

```
1 student = {  
2     "name": "Kim",  
3     "age": 20,  
4     "major": "Physics"  
5 }
```

5.1. 값에 접근하기

```
1 student = {  
2     "name": "Kim",  
3     "age": 20  
4 }  
5  
6 print(student["name"])  
7 print(student["age"])
```

존재하지 않는 키를 대괄호로 조회하면 `KeyError`가 발생한다.

5.2. 추가와 수정

존재하지 않는 키에 값을 대입하면 새 항목이 추가되고, 기존 키에 대입하면 값이 수정된다.

```
1 student = {}  
2  
3 student["name"] = "Kim"  
4 student["age"] = 20  
5 student["age"] = 21  
6  
7 print(student)
```

5.3. 삭제

```
1 student = {  
2     "name": "Kim",  
3     "age": 20  
4 }  
5  
6 del student["age"]  
7 print(student)
```

5.4. 안전한 조회: `get()`

`get()`은 키가 없을 때 오류 대신 `None` 또는 지정한 기본값을 반환한다.

```
1 student = {"name": "Kim"}  
2  
3 print(student.get("name"))  
4 print(student.get("age"))  
5 print(student.get("age", "Not Found"))
```

5.5. 딕셔너리 순회

`items()`는 키와 값을 튜플 형태로 제공한다.

```
1 scores = {
2     "Kim": 95,
3     "Lee": 88,
4     "Park": 76
5 }
6
7 for name, score in scores.items():
8     print(f"{name}: {score}")
```

`keys()`는 모든 키를, `values()`는 모든 값을 제공한다.

```
1 print(scores.keys())
2 print(scores.values())
```

핵심 포인트

딕셔너리의 키는 중복될 수 없다. 같은 키를 다시 대입하면 기존 값이 새로운 값으로 바뀐다.

실습

04_dictionary.py에서 학생 세 명의 점수를 딕셔너리로 저장하고, `items()`를 이용하여 모든 이름과 점수를 출력한다.

6. 집합

집합은 중복을 허용하지 않는 자료형이다. 원소의 고유성만 중요하고 순서는 중요하지 않을 때 사용한다.

```
1 numbers = {1, 2, 2, 3, 3, 3}
2 print(numbers)
```

출력되는 원소의 순서는 고정되지 않을 수 있지만, 중복은 제거된다.

6.1. 리스트의 중복 제거

```
1 students = ["Kim", "Lee", "Kim", "Park"]
2 unique_students = set(students)
3
4 print(unique_students)
```

6.2. 원소 추가와 삭제

```
1 numbers = {1, 2, 3}
2
3 numbers.add(4)
4 numbers.remove(2)
5
6 print(numbers)
```

6.3. 집합 연산

```

1 set_a = {1, 2, 3}
2 set_b = {3, 4, 5}
3
4 print(set_a | set_b)
5 print(set_a & set_b)
6 print(set_a - set_b)

```

연산	기호	의미
합집합		두 집합의 모든 원소
교집합	&	두 집합에 공통인 원소
차집합	-	첫 번째 집합에만 있는 원소

6.4. 소속 검사

```

1 numbers = {10, 20, 30}
2
3 print(20 in numbers)
4 print(50 in numbers)

```

핵심 포인트

집합은 순서를 보장하지 않으므로 인덱싱이나 슬라이싱을 사용할 수 없다.

실습

05_set.py에서 두 집합 {1, 2, 3, 4}와 {3, 4, 5, 6}의 합집합, 교집합, 차집합을 출력한다.

7. 컴프리헨션

컴프리헨션은 반복과 조건을 이용해 새로운 자료구조를 간결하게 만드는 문법이다.

7.1. 리스트 컴프리헨션

다음 두 코드는 같은 결과를 만든다.

```

1 squares = []
2
3 for number in range(1, 6):
4     squares.append(number ** 2)

```

```

1 squares = [number ** 2 for number in range(1, 6)]

```

기본 구조는 다음과 같다.

```

1 result = [expression for variable in iterable]

```

7.2. 조건을 포함한 리스트 컴프리헨션

```

1 multiples = [
2     number
3     for number in range(1, 21)
4     if number % 3 == 0
5 ]
6
7 print(multiples)

```

if 조건을 통과한 원소만 새로운 리스트에 포함된다.

7.3. 조건 표현식 사용

```

1 numbers = [-3, -1, 0, 2, 5]
2
3 result = [
4     number if number >= 0 else 0
5     for number in numbers
6 ]
7
8 print(result)

```

이 경우 if-else는 원소를 선택하는 필터가 아니라 각 원소를 어떤 값으로 변환할지 결정하는 표현식이다.

7.4. 딕셔너리 컴프리헨션

```

1 squares = {
2     number: number ** 2
3     for number in range(1, 6)
4 }
5
6 print(squares)

```

7.5. 집합 컴프리헨션

```

1 squares = {
2     number ** 2
3     for number in range(6)
4 }
5
6 print(squares)

```

체크 질문

[x for x in numbers if x > 0]에서 표현식, 반복 변수, 반복 대상, 조건은 각각 무엇인가?

실습

06_comprehension.py에서 1부터 20까지의 정수 중 짝수의 제곱만 저장하는 리스트를 컴프리헨션으로 작성한다.

8. 자료구조 비교

자료형	순서	중복	수정 가능
리스트	있음	허용	가능
튜플	있음	허용	불가능
딕셔너리	키 기준	키는 불가	가능
집합	보장하지 않음	불가	가능

자료구조를 선택할 때는 데이터의 의미를 먼저 생각해야 한다. 순서가 중요하고 수정이 필요하다면 리스트, 변경되면 안 되는 묶음은 튜플, 이름과 값을 연결할 때는 딕셔너리, 중복 제거와 집합 연산이 필요할 때는 집합이 적합하다.

9. 종합 실습: practice_day03.py

이번 종합 실습에서는 학생 목록과 점수 데이터를 여러 자료구조로 처리한다.

9.1. 학생 목록과 중복 제거

```
1 students = ["Kim", "Lee", "Park", "Choi", "Kim"]
2 unique_students = set(students)
```

9.2. 점수 딕셔너리

```
1 scores = {
2     "Kim": 95,
3     "Lee": 88,
4     "Park": 76,
5     "Choi": 91
6 }
```

9.3. 보너스 점수 계산

```
1 bonus_scores = {
2     name: score + 5
3     for name, score in scores.items()
4 }
```

9.4. 합격자 선별

```
1 passed_students = [
2     name
3     for name, score in bonus_scores.items()
4     if score >= 90
5 ]
```

9.5. 최고 점수와 최고 득점자

```
1 best_score = max(bonus_scores.values())
2
3 best_students = [
4     name
5     for name, score in bonus_scores.items()
6     if score == best_score
7 ]
```

최고 점수와 같은 점수를 가진 모든 학생을 리스트에 저장하므로 동점자가 여러 명이어도 처리할 수 있다.

9.6. 튜플로 결과 묶기

```
1 best_student_info = [
2     (name, best_score)
3     for name in best_students
4 ]
```

9.7. 전체 코드

```
1 students = ["Kim", "Lee", "Park", "Choi", "Kim"]
2 unique_students = set(students)
3
4 scores = {
5     "Kim": 95,
6     "Lee": 88,
7     "Park": 76,
8     "Choi": 91
9 }
10
11 bonus_scores = {
12     name: score + 5
13     for name, score in scores.items()
14 }
15
16 passed_students = [
17     name
18     for name, score in bonus_scores.items()
19     if score >= 90
20 ]
21
22 best_score = max(bonus_scores.values())
23
24 best_students = [
25     name
26     for name, score in bonus_scores.items()
27     if score == best_score
28 ]
29
30 best_student_info = [
31     (name, best_score)
32     for name in best_students
33 ]
34
35 print(unique_students)
36 print(bonus_scores)
37 print(passed_students)
```

```
38 print(best_student_info)
39
40 for name, score in best_student_info:
41     print(f"{name}: {score}")
```

핵심 포인트

최고 득점자를 한 명으로 가정하지 않고 리스트로 저장하면 동점자가 발생해도 코드를 바꿀 필요가 없다.

10. 실습 파일 목록

실습

Day 3의 학습 파일은 다음과 같이 관리한다.

- 01.list.py
- 02.list_methods.py
- 03.tuple.py
- 04.dictionary.py
- 05.set.py
- 06.comprehension.py
- practice_day03.py

11. 핵심 요약

1. 리스트는 순서를 가지며 중복을 허용하고 원소를 수정할 수 있다.
2. 인덱싱은 하나의 원소에 접근하고, 슬라이싱은 구간을 새로운 리스트로 가져온다.
3. `append()`와 `insert()`는 원소를 추가하고, `remove()`와 `pop()`은 원소를 삭제한다.
4. 튜플은 순서를 가지지만 생성 이후 원소를 수정할 수 없다.
5. 튜플 언패킹을 이용하면 여러 값을 여러 변수에 한 번에 대입할 수 있다.
6. 딕셔너리는 키와 값의 쌍으로 데이터를 저장한다.
7. `get()`은 존재하지 않는 키를 안전하게 조회할 수 있다.
8. `items()`는 키와 값을 함께 순회할 때 사용한다.
9. 집합은 중복을 허용하지 않으며 순서를 보장하지 않는다.
10. 집합은 합집합, 교집합, 차집합 연산을 지원한다.
11. 컴프리헨션은 반복과 조건을 이용해 새로운 자료구조를 간결하게 생성한다.
12. 최고 득점자를 리스트로 저장하면 여러 명의 동점자를 함께 처리할 수 있다.